Research Article

Alexandr Udeneev and Oleg Bakhteev

Extracting Inductive Bias Using Generative Models

https://doi.org/10.1515/sample-YYYY-XXXX
Received Month DD, YYYY; revised Month DD, YYYY; accepted Month DD, YYYY

Abstract: The connectivity structure of a neural network influences how well it generalizes from limited data, yet designing this structure often requires domain expertise. We propose a generative multi-task learning framework that learns a distribution over network structures from solutions to related tasks. Our central hypothesis is that structural regularities shared across these solutions provide an inductive bias that transfers to new tasks. We generate structures autoregressively, specifying both connectivity and weight sharing, and train their numerical parameters from scratch on each new task. Experiments on a synthetic signal-detection benchmark show that transferring these structures improves generalization over tuned fully connected baselines.

Keywords: inductive bias, generative model

1 Introduction

Choosing an appropriate inductive bias is crucial for generalization [1]. By constraining the hypothesis space to reflect task structure, it can enable effective learning from limited data. Examples include local connectivity and spatial weight sharing in convolutional networks [2], and recurrent state updates with parameters shared across time [3]. Suitable parameter-sharing patterns also enforce equivariance to task-relevant transformations [4].

Selecting suitable structural biases by hand requires domain expertise, particularly when the relevant properties of a new task are not known in advance [5]. This motivates learning such biases from data. Yeh et al. [5] optimize parameter-sharing schemes to discover equivariances, while Zhou et al. [6] meta-learn sharing patterns across related tasks. Learning groups of similar weights has also been studied through soft weight tying [7] and weight-sharing regularization [8]. Atanov et al. [9] learn generative priors from convolutional filters of previously trained networks and use them to guide learning on new datasets. Schürholt et al. [10] learn generative representations of trained networks to sample new numerical weights. Sun et al. [11] extract task-specific subnetworks by iterative pruning and share parameters where these subnetworks overlap. Ando and Zhang [12] learn a shared predictive structure across tasks while allowing task-specific predictors.

More generally, a single task may admit multiple near-optimal structures, and its data alone need not identify which constraints will transfer to other tasks. Baxter [1] provides a theoretical framework for learning a shared hypothesis space across tasks, with generalization bounds for new tasks from the same environment under suitable capacity and sampling assumptions. Our focus is on transferring connectivity and weight-sharing constraints extracted from trained solutions, with numerical values learned from scratch for each new task.

In this paper, we present a generative multi-task learning framework that learns a distribution over network structures from solutions to related tasks. We represent a structure by an indexator that specifies which connections are present and which weights share a numerical value. The corresponding codebook contains the trainable values. We retain candidate structures according to their validation performance after training a fresh codebook on source tasks.

Our central hypothesis is that structural regularities shared across source solutions provide an inductive bias that transfers to new tasks. We model these regularities with an autoregressive distribution over indexators, using MADE [13]. On a new task, the frozen generator proposes structures, whose numerical parameters are trained from scratch and whose performance is assessed on query data.

On a synthetic benchmark adapted from Dynamic Signal Distribution [14], the extracted structures improve generalization over tuned fully connected models and the same connectivity mask without weight sharing. Simple frequency-based proposals achieve similar accuracy to the autoregressive model. A real-image control using input-selection masks shows no advantage over tuned dense baselines.

Our main contributions are as follows:

– We establish sufficient conditions under which structural search finds low-error solutions that dense gradient-based optimization is unlikely to reach within a fixed number of steps.
– A multi-task framework for transferring connectivity and weight-sharing constraints while learning new numerical parameters.
– A procedure for extracting, validating, and autoregressively generating indexators from source solutions.
– An empirical evaluation of structural recovery and transfer, including simple structural baselines and a real-image control.

2 Related Works

TODO

3 Preliminary

Related prediction tasks may benefit from the same relations between weights even when they require different numerical values. For example, a group of connections may share a coefficient in each task, while its value changes from one task to another. Reusing this pattern constrains the parameters learned on a new task. Such constraints can help a predictor generalize from limited data and change which solutions an optimizer finds within a given budget. We study this form of transfer by separating the structure of a network from its numerical parameters. The structure is proposed from experience with previous tasks; its usefulness is judged after the numerical parameters have been learned anew.

3.1 Multi-task learning

Multi-task learning uses regularities shared by related tasks to improve learning. Here, each task τ is a supervised prediction problem with its own distribution P_τ over input–target pairs $(\mathbf{x}, y)$. Tasks belong to a family $\mathcal{T}$, share input and target spaces $\mathcal{X}$ and $\mathcal{Y}$, and occur according to a distribution $p(\tau)$ [1]. We consider task families in which structural patterns useful for solving one task may also help solve others.

We learn these patterns from source tasks $\mathcal{T}_{\text{tr}}$ and assess their transfer on a disjoint collection of target tasks $\mathcal{T}_{\text{val}}$ drawn from the same task environment. For each task, a *support sample* S_τ^{s} is used to fit numerical parameters, and an independent *query sample* S_τ^{q} is used to compare the fitted predictors [6, 15]. Both consist of independent draws from P_τ. For a predictor f and a sample S, the empirical loss is

$$\mathcal{L}_S(f) := \frac{1}{|S|} \sum_{(\mathbf{x},y)\in S} \ell(f(\mathbf{x}), y). \tag{1}$$

Here, ℓ measures prediction error on one example, such as squared error for regression or cross-entropy for classification.

3.2 Parameter sharing

Parameter sharing assigns a single trainable value to a group of network weights. Deep Compression represents such weights using a *codebook* of shared values and an index for each connection [16]. We use this representation to separate the pattern of shared parameters from the numerical values learned for a particular task.

We fix a common network architecture and collect the weights subject to these constraints into a vector $\mathbf{w} \in \mathbb{R}^P$. An integer array $\mathbf{I} \in \{0, \dots, K\}^P$ assigns each weight to an entry in the codebook $\mathbf{C}_\tau = (C_{\tau,0}, \dots, C_{\tau,K})$. We call this array an *indexator*. The weights are given by

$$w_p = C_{\tau,I_p}, \qquad p = 1, \dots, P, \qquad C_{\tau,0} = 0. \tag{2}$$

Connections with the same positive index therefore share one trainable parameter. We reserve index 0 for a fixed zero value, so that the same representation also specifies which connections are absent. A binary connectivity mask is the special case in which each active connection has its own trainable codebook entry.

For the trainable weights, this is equivalent to the discrete assignment representation of Yeh et al. [5]: each row of a binary assignment matrix selects one shared numerical parameter. The indexator records the selected column for each weight. Its positive labels simply name groups; consistently renaming these labels and the corresponding codebook entries preserves the sharing pattern.

For example,

$$\mathbf{I} = (1, 2, 1, 0) \quad \Longrightarrow \quad \mathbf{w} = (C_{\tau,1}, C_{\tau,2}, C_{\tau,1}, 0).$$

The first and third weights are tied, and the fourth is fixed at zero. One task may learn $\mathbf{w} = (0.7, -1.2, 0.7, 0)$ and another $\mathbf{w} = (-0.4, 0.9, -0.4, 0)$ under the same constraints. The transferable structure is the assignment of connections to shared parameters; the codebook supplies their values on each task.

We write $f_{\mathbf{I},\mathbf{C}_\tau,\boldsymbol{\theta}_\tau}$ for the resulting predictor, where $\boldsymbol{\theta}_\tau$ contains any remaining trainable parameters, such as biases. On a new task, we reuse only the proposed indexator $\mathbf{I}$. In every fitting attempt, $\mathbf{I}$ is held fixed, while all trainable entries of $\mathbf{C}_\tau$ and all of $\boldsymbol{\theta}_\tau$ are initialized afresh and optimized on the task's support sample. This also applies to repeated attempts with the same indexator.

3.3 Good solutions and structural transfer

Let $\widehat{f}_{\tau,\mathbf{I}}$ be a predictor obtained by fitting numerical parameters under structure $\mathbf{I}$. For a chosen error tolerance ε, we call it a *good solution* to task τ when its expected loss on new examples satisfies

$$\mathcal{R}_\tau(\widehat{f}_{\tau,\mathbf{I}}) := \mathbb{E}_{(\mathbf{x},y)\sim P_\tau}\left[\ell(\widehat{f}_{\tau,\mathbf{I}}(\mathbf{x}), y)\right] \le \varepsilon. \tag{3}$$

The tolerance specifies the desired prediction quality. Whether training reaches it also depends on the support data, optimizer, initialization rule, and training budget. A structure may admit a good solution while the numerical optimizer rarely finds it. We therefore assess structures through the outcomes of actual fitting attempts under specified conditions, including fresh initializations.

Several structures may support good solutions to a task, and useful patterns may recur across tasks. Our shared learned object is a distribution $Q_{\boldsymbol{\phi}}(\mathbf{I})$ over indexators. Source tasks are used to learn its parameters $\boldsymbol{\phi}$. On a target task, $\boldsymbol{\phi}$ is frozen and the model proposes candidate structures, each of which receives fresh numerical training. Successful transfer requires both proposing a suitable structure and fitting its parameters well.

Since expected loss is usually unknown, we select among the fitted predictors by their query loss and measure final performance on further test examples unused in fitting or selection. We evaluate the resulting procedure across new tasks and independent runs, accounting for all candidate fits and restarts in the target-task training budget and reporting source-task learning costs separately. The aim is to make good solutions more reliably attainable with the available data and computation; comparisons with dense optimization establish whether the learned constraints provide that benefit.

4 Method

TODO test

5 Experiments

TODO

6 Conclusion

TODO

References

TODO

[1] Jonathan Baxter. A model of inductive bias learning. *Journal of artificial intelligence research*, 12:149–198, 2000.

[2] Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998. 10.1109/5.726791.

[3] Jeffrey L. Elman. Finding structure in time. *Cognitive Science*, 14(2):179–211, 1990. 10.1207/s15516709cog1402_1.

[4] Siamak Ravanbakhsh, Jeff Schneider, and Barnabás Póczos. Equivariance through parameter-sharing. In *Proceedings of the 34th International Conference on Machine Learning*, volume 70 of *Proceedings of Machine Learning Research*, pages 2892–2901. PMLR, 2017. URL https://proceedings.mlr.press/v70/ravanbakhsh17a.html.

[5] Raymond A Yeh, Yuan-Ting Hu, Mark Hasegawa-Johnson, and Alexander Schwing. Equivariance discovery by learned parameter-sharing. In *International Conference on Artificial Intelligence and Statistics*, pages 1527–1545. PMLR, 2022.

[6] Allan Zhou, Tom Knowles, and Chelsea Finn. Meta-learning symmetries by reparameterization. In *International Conference on Learning Representations*, 2021. URL https://arxiv.org/abs/2007.02933.

[7] Steven J. Nowlan and Geoffrey E. Hinton. Adaptive soft weight tying using gaussian mixtures. In *Advances in Neural Information Processing Systems*, volume 4, pages 993–1000, 1991. URL https://proceedings.neurips.cc/paper/1991/hash/05f971b5ec196b8c65b75d2ef8267331-Abstract.html.

[8] Mehran Shakerinava, Motahareh Sohrabi, Siamak Ravanbakhsh, and Simon Lacoste-Julien. Weight-sharing regularization. In *Proceedings of The 27th International Conference on Artificial Intelligence and Statistics*, volume 238 of *Proceedings of Machine Learning Research*, pages 4204–4212. PMLR, 2024. URL https://proceedings.mlr.press/v238/shakerinava24a.html.

[9] Andrei Atanov, Arsenii Ashukha, Kirill Struminsky, Dmitry Vetrov, and Max Welling. The deep weight prior. In *International Conference on Learning Representations*, 2019. URL https://arxiv.org/abs/1810.06943.

[10] Konstantin Schürholt, Boris Knyazev, Xavier Giró-i Nieto, and Damian Borth. Hyper-representations as generative models: Sampling unseen neural network weights. In *Advances in Neural Information Processing Systems*, volume 35, 2022. URL https://arxiv.org/abs/2209.14733.

[11] Tianxiang Sun, Yunfan Shao, Xiaonan Li, Pengfei Liu, Hang Yan, Xipeng Qiu, and Xuanjing Huang. Learning sparse sharing architectures for multiple tasks. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 34, pages 8936–8943, 2020. 10.1609/aaai.v34i05.6424. URL https://ojs.aaai.org/index.php/AAAI/article/view/6424.

[12] Rie Kubota Ando and Tong Zhang. A framework for learning predictive structures from multiple tasks and unlabeled data. *Journal of Machine Learning Research*, 6(61):1817–1853, 2005. URL https://jmlr.org/papers/v6/ando05a.html.

[13] Mathieu Germain, Karol Gregor, Iain Murray, and Hugo Larochelle. MADE: Masked autoencoder for distribution estimation. In *Proceedings of the 32nd International Conference on Machine Learning*, volume 37 of *Proceedings of Machine Learning Research*, pages 881–889. PMLR, 2015. URL https://proceedings.mlr.press/v37/germain15.html.

[14] Aakash Sunil Lahoti, Stefani Karp, Ezra Winston, Aarti Singh, and Yuanzhi Li. Role of locality and weight sharing in image-based tasks: A sample complexity separation between CNNs, LCNs, and FCNs. In *International Conference on Learning Representations*, 2024. URL https://proceedings.iclr.cc/paper_files/paper/2024/hash/71b17f00017da0d73823ccf7fbce2d4f-Abstract-Conference.html.

[15] Chelsea Finn, Pieter Abbeel, and Sergey Levine. Model-agnostic meta-learning for fast adaptation of deep networks. In *Proceedings of the 34th International Conference on Machine Learning*, volume 70 of *Proceedings of Machine Learning Research*, pages 1126–1135. PMLR, 2017. URL https://proceedings.mlr.press/v70/finn17a.html.

[16] Song Han, Huizi Mao, and William J. Dally. Deep Compression: Compressing deep neural networks with pruning, trained quantization and huffman coding. In *International Conference on Learning Representations*, 2016. URL https://arxiv.org/abs/1510.00149.